LAUNCHPATH AI

Implementation Blueprint

Days 2–10 | 2–3 hours per day

Your Intelligent Placement Readiness Companion

CAPSTONE PROJECT • INDIA • ZERO-BUDGET MVP

Contents

• Day 2 — Project foundation and UI shell

• Day 3 — Onboarding, profile and PDF resume

• Day 4 — Role discovery, adaptive assessment and readiness engine

• Day 5 — Weekly roadmap, learning library and AI tutor

• Day 6 — Aptitude and coding practice

• Day 7 — Interview simulator and evaluation

• Day 8 — Job discovery, application tracker and calendar

• Day 9 — Analytics, PDF reports, integration and QA

• Day 10 — Deployment, hardening, testing and final demo

• Execution rules and release gates

Planning assumptions

Day 1 is complete or the repository exists. Each remaining day is capped at roughly 2–3 hours. Build the integrated critical path first; defer stretch goals rather than sacrificing testing.

Day 2 — Project foundation and UI shell

Objective / expected outcome: A runnable application skeleton with responsive navigation, shared design system and clearly labelled demo mode.

Step-by-step tasks

• Inspect repository, package scripts, framework and current routes before changing anything.

• Create or verify app shell, route structure and shared layout components.

• Define design tokens: typography, spacing, colors, radii, shadows and breakpoints.

• Build responsive sidebar/topbar navigation for Dashboard, Profile, Roles, Roadmap, Practice, Interviews, Jobs and Reports.

• Create reusable Button, Card, Badge, Progress, EmptyState, Loading and Error components.

• Add fictional guest dashboard preview with explicit 'Sample data' label.

• Add README with install, run, build and environment setup.

Deliverables & definition of done

• App boots locally with one documented command.

• Navigation works on mobile and desktop; active route is visible.

• No console errors; sample data is clearly fictional.

• Shared components are reused rather than duplicated.

Testing checklist

• App boots locally with one documented command.

• Navigation works on mobile and desktop; active route is visible.

• No console errors; sample data is clearly fictional.

• Shared components are reused rather than duplicated.

Stretch goals (only if time remains)

• Keyboard navigation and focus states.

• Dark/light theme toggle only if core shell is complete.

Common risks & troubleshooting

• Route mismatch: inspect framework router before adding dependencies. CSS conflicts: centralize tokens. Blank page: check console, entry point and import casing.

Fresh-conversation implementation prompt

Project: LaunchPath AI, a zero-budget-first placement readiness companion for final-year students and fresher job seekers in India. Build only Day 2: a runnable responsive UI shell. Stack is whatever already exists in this repository; do not replace it or add dependencies without need. Inspect all existing files, package scripts, routes and styles before editing. Preserve working functionality and existing conventions. Create a professional corporate shell with responsive navigation for Dashboard, Profile, Roles, Roadmap, Practice, Interviews, Jobs, Reports and Settings; reusable UI primitives; design tokens; and a fictional guest dashboard preview clearly marked SAMPLE DATA. Do not implement backend/auth/AI yet. First report repository findings and a small plan, then make the smallest coherent changes. Give exact terminal commands to install/run/build, identify files changed, and provide manual verification steps. Do not claim it works until I run it. Stop and wait for my verification before proceeding.

Timebox: 15 min inspect/plan • 90–110 min build • 25–35 min test and document.

Day 3 — Onboarding, profile and PDF resume

Objective / expected outcome: Users can enter profile data manually or upload a PDF, review extracted text and correct fields.

Step-by-step tasks

• Inspect Day 2 result and current code; run the app before editing.

• Create onboarding steps for education, skills, projects, experience, target preferences and available weekly hours.

• Implement profile form with validation and autosave/local persistence appropriate to current stack.

• Add PDF upload with type/size validation and extraction preview using an available open-source library only if compatible.

• Map extracted content into editable draft fields; never auto-assert uncertain facts.

• Provide manual-entry fallback when parsing fails or file is scanned.

• Add consent explanation before any external analysis; no external AI call in this day.

Deliverables & definition of done

• Manual profile can be saved, refreshed and edited.

• Valid PDF shows extracted text or a clear parse status.

• Parse failure presents manual entry without blocking onboarding.

• User confirms extracted data before it becomes profile data.

Testing checklist

• Manual profile can be saved, refreshed and edited.

• Valid PDF shows extracted text or a clear parse status.

• Parse failure presents manual entry without blocking onboarding.

• User confirms extracted data before it becomes profile data.

Stretch goals (only if time remains)

• Add profile completeness indicator.

• Add sample resume fixture for local testing.

Common risks & troubleshooting

• Scanned PDFs may yield no text: explain limitation and offer manual input. Large file: enforce size limit. Browser parser issues: isolate parser and keep form usable.

Fresh-conversation implementation prompt

Project: LaunchPath AI is a zero-budget-first placement readiness companion for final-year students and fresher job seekers in India. Current milestone: Day 3 onboarding, profile and PDF resume. Inspect the existing repository and current working UI before editing; preserve Day 2 navigation, design tokens and demo mode. Do not rewrite the app or change frameworks. Build a multi-step onboarding/profile form for education, skills, projects, experience, links, target preferences and weekly availability. Add PDF upload with file type/size validation and text extraction using an existing compatible open-source dependency if practical. Show extracted text and editable suggested fields; require user confirmation. If parsing fails or PDF is scanned, offer manual entry and continue. Do not call an external AI service. Persist using the project's current storage approach and clearly label local-only persistence. Give exact commands, changed-file list, test cases and expected results. Ask me to run the app and verify before making further changes.

Timebox: 15 min inspect/plan • 90–110 min build • 25–35 min test and document.

Day 4 — Role discovery, adaptive assessment and readiness engine

Objective / expected outcome: A role-fit report, role-based diagnostic and explainable readiness score with coverage.

Step-by-step tasks

• Inspect current code and confirm profile data shape before editing.

• Define a small curated role catalog and role-to-skill mapping for 3–5 entry-level roles.

• Allow manual role choice: one primary and up to two backups.

• Generate top-three role recommendations from transparent profile-skill overlap; show evidence and gaps.

• Create role-tagged diagnostic questions with topic and difficulty metadata.

• Implement simple adaptive branching: correct → harder/adjacent; incorrect → prerequisite or explanatory item.

• Implement score service with agreed weights and missing-data renormalization.

• Show overall /100, category scores, coverage, timestamp and next actions; include non-hiring-probability disclaimer.

Deliverables & definition of done

• Role selection enforces one primary and max two backups.

• Top-three report explains fit evidence and missing skills.

• Quiz adapts at least one follow-up based on answer.

• Score weights sum to 100%; missing categories are Not assessed, not silently zero.

Testing checklist

• Role selection enforces one primary and max two backups.

• Top-three report explains fit evidence and missing skills.

• Quiz adapts at least one follow-up based on answer.

• Score weights sum to 100%; missing categories are Not assessed, not silently zero.

Stretch goals (only if time remains)

• Add score snapshot history.

• Add role comparison view.

Common risks & troubleshooting

• Score inflation: cap repeated practice signals. Sparse profile: request more evidence. Adaptive quiz bug: unit-test branching and answer state.

Fresh-conversation implementation prompt

Project: LaunchPath AI serves final-year students and fresher job seekers in India. Confirmed rules: one primary role plus up to two backup roles; manual selection and AI-style recommendations; top three role-fit reports; resume/profile plus role-based adaptive diagnostic; explainable hybrid readiness score /100 with category scores; continuous updates and weekly reassessment. Inspect the current repository, profile schema and working flows before editing. Preserve existing behavior. Build a small transparent role catalog (3–5 realistic entry-level roles), role-skill mappings, manual role selection and top-three recommendations with evidence/gaps. Add a role-tagged diagnostic and deterministic adaptive branching. Implement readiness calculation with weights: profile/resume 20%, diagnostic 30%, aptitude 15%, coding 20%, interview 10%, consistency 5%. If evidence is missing, mark category Not assessed and renormalize over assessed categories; show coverage and timestamp. Clearly state the score is not hiring probability. Use deterministic logic when AI is unavailable; do not add paid APIs. Provide exact commands, files changed, tests for weights/missing data/adaptation, and wait for my verification.

Timebox: 15 min inspect/plan • 90–110 min build • 25–35 min test and document.

Day 5 — Weekly roadmap, learning library and AI tutor

Objective / expected outcome: A role-aware weekly plan, curated resources and beginner-first tutor with deterministic fallback.

Step-by-step tasks

• Inspect existing score/role/profile structures and reuse them.

• Create stable weekly milestone templates for selected roles.

• Generate tasks from gaps, available hours and user level; keep milestone stable while adapting tasks.

• Build resource catalog with topic, level, format, estimated time and source URL.

• Create beginner-first tutor interface: concept → example → implementation → practice check.

• If AI integration is unavailable or consent is declined, use curated explanations/templates.

• Store topic progress and learning notes minimally; expose reset/delete controls.

Deliverables & definition of done

• Roadmap shows current week, milestone and actionable tasks.

• Task completion persists and updates progress.

• Resources are filterable by topic/level and have source labels.

• Tutor gives explanation, example and practice check without paid API.

Testing checklist

• Roadmap shows current week, milestone and actionable tasks.

• Task completion persists and updates progress.

• Resources are filterable by topic/level and have source labels.

• Tutor gives explanation, example and practice check without paid API.

Stretch goals (only if time remains)

• Add weekly reassessment reminder in-app.

• Add resource bookmarks.

Common risks & troubleshooting

• Generic roadmap: verify role-gap mapping. Broken resource links: validate URLs manually. AI output inconsistency: enforce structured output and fallback.

Fresh-conversation implementation prompt

Project: LaunchPath AI is a zero-budget-first placement companion. Current milestone: Day 5 roadmap, learning library and tutor. Inspect existing code, role catalog, readiness categories and data model before editing; preserve working features. Build stable weekly milestones for the selected primary role and adapt task ordering/difficulty from skill gaps, weekly hours and progress. Do not reshuffle the whole plan on every score update. Add a curated learning resource library with topic, level, type, estimated time and source link. Add a beginner-first tutor flow that explains a concept, gives a worked example, then a small practice check. Topic-based learning progress may be stored, but avoid retaining unnecessary raw chat history. External AI must be optional and require explicit purpose-specific consent; provide curated/template fallback when unavailable or declined. No paid API dependency. Give exact commands, changed files, test checklist and wait for my verification.

Timebox: 15 min inspect/plan • 90–110 min build • 25–35 min test and document.

Day 6 — Aptitude and coding practice

Objective / expected outcome: Working quantitative/logical practice and DSA/fundamentals practice with hints and explanations.

Step-by-step tasks

• Inspect current practice and persistence patterns.

• Create a small question bank for quantitative aptitude and logical reasoning with answers and explanations.

• Implement practice sessions, attempts, topic accuracy and review state.

• Add coding fundamentals/DSA problem cards with constraints, examples, hints and reference solutions.

• Show time/space complexity explanations and common edge cases.

• Optional lightweight editor only if core practice is complete; do not build a judge.

• Use sample tests or self-reported completion and label validation limits.

Deliverables & definition of done

• Aptitude answer is checked and explained.

• Attempt history and topic accuracy update.

• Coding problems reveal progressive hints and reference solution.

• Complexity and edge-case explanation is visible; no claim of judge-based correctness.

Testing checklist

• Aptitude answer is checked and explained.

• Attempt history and topic accuracy update.

• Coding problems reveal progressive hints and reference solution.

• Complexity and edge-case explanation is visible; no claim of judge-based correctness.

Stretch goals (only if time remains)

• Add spaced-review queue.

• Add filters by difficulty.

Common risks & troubleshooting

• Ambiguous answer keys: manually test fixtures. No execution environment: keep editor optional and disclose self-check. State resets: persist attempts.

Fresh-conversation implementation prompt

Project: LaunchPath AI supports final-year students and freshers. Day 6 scope is aptitude and coding practice only. Inspect the existing app, question schemas and storage before editing; preserve all working flows. Add a small curated quantitative aptitude and logical reasoning bank with answer validation, worked explanations, attempts and topic accuracy. Add beginner-friendly programming fundamentals and DSA problems with examples, constraints, progressive hints, reference solution, complexity analysis and edge cases. A lightweight editor is optional only after these flows work; do not build a full coding judge or claim arbitrary code is validated. If no execution service exists, use sample test cases/self-check and disclose that limitation. Keep all content locally available and usable without paid APIs. Provide exact commands, changed files, a manual test checklist including wrong answers and refresh persistence, and wait for my verification.

Timebox: 15 min inspect/plan • 90–110 min build • 25–35 min test and document.

Day 7 — Interview simulator and evaluation

Objective / expected outcome: Text-first HR/technical mock interviews with presets and rubric feedback.

Step-by-step tasks

• Inspect current interview UI and data model.

• Create interview presets: Quick 5, Standard 10, Deep 15–20 and Custom.

• Use curated HR and technical questions tagged by role/topic/difficulty; optional AI follow-ups only with consent.

• Implement text answer capture and session progress; voice only if browser support is reliable and permission is explicit.

• Score with a transparent rubric: relevance, structure, correctness, clarity and examples.

• Return evidence-based feedback, one strength, one improvement and actionable practice task.

• Avoid scoring personality, accent, emotion or protected traits.

Deliverables & definition of done

• User can start and finish a text interview using each preset.

• Questions match selected role and interview type.

• Rubric dimensions and feedback are visible with rationale.

• Voice unsupported cleanly falls back to text.

Testing checklist

• User can start and finish a text interview using each preset.

• Questions match selected role and interview type.

• Rubric dimensions and feedback are visible with rationale.

• Voice unsupported cleanly falls back to text.

Stretch goals (only if time remains)

• Add session history and comparison.

• Add downloadable feedback summary.

Common risks & troubleshooting

• AI overclaims: constrain to rubric evidence. Browser speech API variability: feature-detect and preserve text mode. Long sessions: autosave.

Fresh-conversation implementation prompt

Project: LaunchPath AI is a zero-budget placement preparation companion. Day 7: build an interview simulator. Inspect existing app, role/question data and persistence before editing; do not break previous features. Implement HR and technical text interviews with Quick 5, Standard 10, Deep 15–20 and Custom presets. Use a curated question bank and role/topic tags. AI-generated follow-ups are optional, require explicit consent for each external analysis and must have a curated fallback. Add browser-supported voice only if reliable feature detection and permission handling are possible; text must always work. Evaluate answers using transparent rubric dimensions: relevance, structure, correctness, clarity and examples. Give explanations tied to answer evidence and one actionable improvement task. Do not infer personality, emotion, accent quality or protected characteristics. No hiring prediction. Provide exact commands, changed files, test cases for each preset and fallback, then wait for my verification.

Timebox: 15 min inspect/plan • 90–110 min build • 25–35 min test and document.

Day 8 — Job discovery, application tracker and calendar

Objective / expected outcome: A safe job discovery fallback, full application status tracker and date-based calendar.

Step-by-step tasks

• Inspect current data model and navigation before editing.

• Add job cards with title, company, location, role, source and last-checked date.

• Use live listings only if a reliable permitted free source is configured; otherwise label curated/demo jobs clearly.

• Allow manual job creation and edits.

• Implement statuses: Saved, Applied, Assessment, Interview, Offer, Rejected, Withdrawn.

• Add application dates, deadlines, notes and next action.

• Build calendar/list view and in-app reminders; no SMS/WhatsApp.

Deliverables & definition of done

• Manual job can be added and tracked through all statuses.

• Status changes persist and are reflected in dashboard.

• Calendar displays due dates and reminders.

• Demo listings are unmistakably labelled; no fabricated live feed.

Testing checklist

• Manual job can be added and tracked through all statuses.

• Status changes persist and are reflected in dashboard.

• Calendar displays due dates and reminders.

• Demo listings are unmistakably labelled; no fabricated live feed.

Stretch goals (only if time remains)

• Add CSV export.

• Add filters for role/status/location.

Common risks & troubleshooting

• Feed unavailable: default to manual mode. Date/time bugs: store ISO timestamps and render local timezone. Reminder permission denied: show in-app notice.

Fresh-conversation implementation prompt

Project: LaunchPath AI is a zero-budget-first placement companion. Day 8 scope: job discovery, application tracking and calendar. Inspect existing routes, data models and storage before editing; preserve all working features. Build job cards with source and checked date. Only integrate live listings if a reliable, permitted free source is already available and terms allow it; otherwise provide clearly labelled curated demo jobs and manual job entry. Implement exactly these statuses: Saved, Applied, Assessment, Interview, Offer, Rejected, Withdrawn. Add application dates, deadlines, notes and next action. Add calendar/list view and in-app reminders only; do not add SMS or WhatsApp. Handle timezone consistently and show permission/persistence limitations. Provide exact commands, files changed and tests for every status transition, manual job creation, date rendering and refresh persistence. Wait for my verification.

Timebox: 15 min inspect/plan • 90–110 min build • 25–35 min test and document.

Day 9 — Analytics, PDF reports, integration and QA

Objective / expected outcome: A cohesive dashboard and downloadable report, with cross-feature integration and critical-path QA.

Step-by-step tasks

• Inspect all modules and list integration gaps before editing.

• Create analytics from real stored events: readiness snapshots, category trends, tasks, practice, interviews and applications.

• Avoid vanity metrics and label sample data.

• Build PDF report export with profile/role, score methodology, progress, practice/interview highlights, applications and next-week plan.

• Integrate dashboard cards with real feature data and meaningful empty states.

• Run end-to-end critical path, responsive, accessibility and fallback tests.

• Fix P0 defects; freeze new features after this day.

Deliverables & definition of done

• Dashboard numbers match source records.

• PDF opens and includes date, methodology and correct user data.

• No sample/demo values presented as real user metrics.

• Critical-path tests pass; errors are actionable.

Testing checklist

• Dashboard numbers match source records.

• PDF opens and includes date, methodology and correct user data.

• No sample/demo values presented as real user metrics.

• Critical-path tests pass; errors are actionable.

Stretch goals (only if time remains)

• Add CSV data export.

• Add print stylesheet.

Common risks & troubleshooting

• Mismatched analytics: derive from canonical records. PDF font/layout overflow: test multi-page data. Time pressure: prioritize P0 defects and freeze features.

Fresh-conversation implementation prompt

Project: LaunchPath AI is a zero-budget placement companion. Day 9 is integration, analytics, PDF reporting and QA. Inspect the full repository and current feature behavior first. Produce a gap list before editing. Preserve working functionality and avoid adding new product scope. Build dashboard analytics from canonical stored data only: readiness snapshots, category scores, roadmap completion, practice attempts, interviews and application statuses. Clearly label sample data. Implement a professional multi-page PDF report containing profile/target role summary, score methodology and coverage, progress, practice/interview highlights, applications and next-week recommendations. Ensure report values match app records. Run critical-path, mobile responsiveness, keyboard/accessibility, AI-off, PDF-failure, voice-unsupported, jobs-feed-unavailable and local-mode checks. Fix P0 bugs and freeze features. Give exact commands, changed files, test outcomes and known limitations; wait for my verification.

Timebox: 15 min inspect/plan • 90–110 min build • 25–35 min test and document.

Day 10 — Deployment, hardening, testing and final demo

Objective / expected outcome: A public free-tier deployment or reproducible local demo, hardened flows and a concise final walkthrough.

Step-by-step tasks

• Inspect build, environment variables, deployment configuration and current QA findings.

• Run clean install, lint/typecheck (if configured), tests and production build.

• Remove secrets from source; configure environment variables only in host settings.

• Verify auth/data access rules, consent gates, upload validation and error handling.

• Deploy to a free host only after checking current limits/terms; keep local fallback instructions.

• Run smoke tests on deployed URL and mobile viewport; verify no demo data leaks.

• Prepare a 3–5 minute demo script, seeded fictional account and known-limitations slide/README.

Deliverables & definition of done

• Production build succeeds from clean install.

• Public URL works or local fallback is reproducible.

• No secrets in repository/client bundle.

• Core journey and fallback tests pass on deployment.

• Demo script accurately distinguishes implemented features from future work.

Testing checklist

• Production build succeeds from clean install.

• Public URL works or local fallback is reproducible.

• No secrets in repository/client bundle.

• Core journey and fallback tests pass on deployment.

• Demo script accurately distinguishes implemented features from future work.

Stretch goals (only if time remains)

• Add basic monitoring/error logging if free and privacy-safe.

• Record a short demo video if time remains.

Common risks & troubleshooting

• Free-tier restrictions: use local demo. Environment mismatch: document variables. Deployment-only bug: compare build/runtime logs and test a clean browser session.

Fresh-conversation implementation prompt

Project: LaunchPath AI is a zero-budget-first placement readiness companion for final-year students and fresher job seekers in India. Day 10 is hardening and final demo, not feature expansion. Inspect the complete repository, build scripts, environment variables, deployment config and Day 9 QA notes before editing. Preserve working functionality. Run clean install, tests, lint/typecheck if configured and production build. Check secrets, auth/data ownership, consent gates, upload validation, error states and mobile responsiveness. Deploy to a free public host only if current free-tier terms and limits permit; otherwise provide a reproducible local demo fallback. Smoke-test the deployed URL or local build through the full journey: profile → role → diagnostic → readiness → roadmap → practice → interview → application tracker → PDF report. Confirm AI-off and unavailable-job-feed fallbacks. Prepare a 3–5 minute demo script, fictional demo account/data and honest limitations. Give exact commands, deployment steps, changed files and verification checklist. Do not claim deployment succeeded until I confirm the URL works; stop and wait for my verification.

Timebox: 15 min inspect/plan • 90–110 min build • 25–35 min test and document.

Execution rules and release gates

• Inspect before every change; preserve working behavior.

• Use small, reversible changes and keep a working checkpoint each day.

• No paid API dependency; external AI requires explicit purpose-specific consent.

• Day 9 freezes features; Day 10 is for hardening, deployment and demo.

• Definition of done requires verification in the user's environment.